

ตรวจทิศทาง Murakkab reproduction — 2026-09-12

เป้าหมายที่ใช้ตัดสิน

สร้าง Murakkab เพื่อเข้าใจระบบต้นฉบับจากการลงมือทำ แล้วปรับปรุง component ภายในสถาปัตยกรรมเดิมเพื่อเปรียบเทียบอย่างเป็นธรรม ไม่ตั้งสมมติฐานล่วงหน้าว่าระบบต้นฉบับผิด และไม่ถือว่าการตรวจ Appendix A.5 เพียงอย่างเดียวเท่ากับการสร้าง Murakkab ครบระบบ

เอกสารนี้เป็นผล review ไม่ใช่การแก้ implementation หรือการอนุมัติ milestone ถัดไป

หลักฐานและขอบเขต

- OSDI PDF ที่ผู้ใช้ส่ง: C:/Users/darkn/Downloads/osdi26-chaudhry (1).pdf โดยเฉพาะ §2.2, §3.1–3.4, Table 1, §4.1–4.8 และ Appendix A.2–A.5 ตรวจภาพ Figure 12, Table 5 และสมการด้วย
- arXiv v2 ที่ผู้ใช้ส่ง: C:/Users/darkn/Downloads/2508.18298v2.pdf โดยเฉพาะ §3, §4.3–4.6 และ Appendix A.5
- README, CLAUDE, PROGRESS, design decisions, โค้ดเส้นทาง development/profile/MILP/reporting และ tests ที่เกี่ยวข้อง
- รัน test suite จริง: 510 passed, 19.20 s บน bundled Python และ pytest 9.1.1 / PuLP 3.3.2 ที่ติดตั้งแยกไว้ใน tmp/audit-deps มี warnings จาก dependencies จำนวนมาก
- ตรวจพฤติกรรมเพิ่มเติมด้วย tmp/murakkab_context_audit.py ซึ่งไม่แก้ production modules การปรับ admissible pairs ใน probe เกิดในหน่วยความจำของการทดลองเท่านั้น
- ไม่ได้เห็น implementation ภายในของผู้เขียน paper จึงแยกค่ากล่าวของ paper ออกจากสิ่งที่พิสูจน์ได้ใน reproduction นี้

คำตัดสิน

โครงการมีฐานวิศวกรรมที่ใช้ต่อได้: separation ของ phases, mock interface, type checking, provenance, missing-data ledger และ MILP backend ที่เปลี่ยนได้ แต่ทิศทางเอกสารและ tests

หลายส่วนเอนจากการสร้างระบบต้นฉบับไปเป็นการพิสูจน์ข้อบกพร่องของการอ่าน A.5 แบบจำกัดที่สุด

สถานะที่เหมาะสมคือ partial architecture reproduction พร้อม Appendix A.5 audit harness ยังไม่ใช่ end-to-end Murakkab baseline ที่พร้อมใช้วัด improvement ทั้งระบบ

Milestone 6/7 ยังไม่เริ่มตาม PROGRESS; ยังไม่มี execution package, registry, executable-plan materialization, serving runtime และ auto-scaler ที่ต่อเป็นวงจรจริง

Murakkab ตาม paper: contract ของแต่ละ phase

Phase	Component	หน้าที่และหลักฐาน
Development	Declarative specification	ระบุ task และ dependency; รองรับ developer execution preferences; §3.2
Development	Executor library	LLM, composition, tool พร้อม description/interface/knobs; §3.2
Development	Orchestrator	เลือก executor ด้วย LLM, ตรวจ types, feedback/retry, ส่งกลับให้ developer เมื่อแก้ไม่ได้; §3.2
Development	Logical workflow	DAG ที่ยังไม่ผูก request payload และยังไม่กำหนด hardware/model deployment ขั้นสุดท้าย; §3.2
Optimization	Workflow profiling	วัดคุณภาพและโหลดระดับ executor รวม prompt/completion tokens ภายได้ configurations; §3.3

Phase	Component	หน้าที่และหลักฐาน
Optimization	Model profiling	วัด latency/throughput/energy/cost ตาม model, hardware, parallelism และ load levels; §3.3
Optimization	Demand forecast	ใช้ประวัติทำนาย epoch หน้า; epoch ปกติ 60 นาที; EWMA coefficient 0.5 ใน sensitivity study §4.7
Optimization	MILP	เลือก workflow knobs, model/tool ต่อ executor, instance counts และ routing map; §3.3.1
Optimization	Deployment	สร้าง executable workflows และบันทึก registry; §3.3–3.4
Execution	Dispatch	รับ workflow/query/input/SLO แล้วเลือก executable workflow; §3.4
Execution	Dynamic composition	ประกอบงานจาก executors หรือ workflows ที่มีอยู่; §3.4
Execution	Scheduling/batching	รันตาม dependencies และแชร์ instances; Table 1, Figure 12
Execution	Auto-scaler	สังเกต load ช่วงสั้น, scale, spare capacity และ early re-optimization; §3.4

A.5 ไม่แจกแจงขั้นตอนทั้งหมดนี้ โดยเฉพาะ CPU placement และ DAG scheduling

ความไม่ครบของสมการเป็นข้อจำกัดการสร้างซ้ำที่ตรวจสอบได้ แต่ไม่พิสูจน์ว่า runtime ของผู้เขียนไม่มีพฤติกรรมที่รายงานไว้

ข้อค้นพบจากโค้ดและการทดลอง

R1 — จำนวน GPU ในรายงานนับผิด

optimization/milp/report.py:81 และ optimization/milp/joint.py:349 คำนวณ $\text{sum}(n_m)$ จาก property total_gpus แต่ A.5 นิยาม n_m เป็น model instances และ g_m เป็น parallelism ดังนั้น GPU ต้องนับ $\text{sum}(n_m * g_m)$ ซึ่งในโค้ดนี้ $g_m = m.tp$

Probe: derived_tiers, epoch 0, ENERGY objective, default 70/30 SLO mix:

Arm	จำนวนที่ property เรียก GPUs	GPU ตาม allocation จริง
Separate by workflow	131 instances	418 GPUs
Joint, $\mu=1$	131 instances	418 GPUs
Joint, $\mu=0.784$	102 instances	326 GPUs

22.1374% เป็น reduction ของ instance count; reduction ของ GPU ในรันนี้คือประมาณ 22.01% ตัวเลขยังมาจาก control ที่เปลี่ยน latency tiers และ fitted μ จึงไม่ใช่ reproduction ของ Table 2

R2 — Cost budget ใช้หน่วยเวลาคนละฐาน

optimization/milp/__init__.py:94 และ joint.py:392 นำ objective COST ที่คูณ EPOCH_SECONDS แล้วมาเป็น RHS ของ Eq.6 แต่ LHS ของ Eq.6 เป็น dollars/second

Probe: $O2 = \$244.773$ ต่อ epoch; multiplier 1.25 ให้ $\$305.96625$ แต่ถูกใช้เป็น 305.96625 dollars/second แทน 0.084990625 dollars/second ทำให้งบหลวมขึ้น 3,600 เท่าหากตั้งใจใช้ budget ต่อ epoch เดียวกัน

นี่เป็น implementation bug ไม่ใช่ข้อบกพร่องของ paper และแยกจากความไม่ชัดเจนระหว่าง consumed-work cost กับ provisioned cost ใน A.5

R3 — Operating points ถูกยุบโดยไม่จำเป็น และบางจุดสูญหายจาก sensitivity path

optimization/profiles/schema.py:216 จำกัด profile key เป็น model/GPU/TP และประกาศว่าการเพิ่ม operating-point index เป็นการขอม paper แต่ §3.3 ระบุ profiles across load levels และ A.2/A.3 ระบุการเพิ่ม allowed load เพื่อเพิ่ม batching ตาม SLO

การให้แต่ละ measured operating point เป็น candidate profile เป็น reconstruction ที่มีเหตุผลรองรับ ไม่ใช่สิ่งที่ A.5 ห้ามไว้ แม้ paper จะไม่ได้แจกแจง encoding นี้โดยตรง

model_profiles.py:175 เลือกหนึ่ง table point ก่อนประกอบ curve กรณี Llava/H100/TP4 ไม่มี figure curve มาชัดเจน จึงเหลือเพียง TPS=2836, TPOT=0.007; TABLE_REPORTED, MAX_THROUGHPUT และ KNEE ให้ 2836 เท่ากันทั้งหมด ทั้งที่ Table 5 มี 479 และ 3271 TPS ด้วย ข้ออ้าง sensitivity 479→3271 ในเอกสารไม่เกิดขึ้นใน implementation ปัจจุบัน

R4 — TTFT ประกอบจากคนละ operating point

model_profiles.py:305 เลือก TTFT แรกที่อ่านได้จาก curve แล้วนำมารวมกับ throughput/TPOT ของ table row

Probe Gemma/A100/TP4: point ที่เลือก TPS=700 ใช้ TTFT=0.39916 จาก curve ที่ TPS=556.1 ขณะที่จุดใกล้ 700 ที่ TPS=699.58 มี TTFT=0.40844

ควรจับคู่หรือ interpolate ที่ throughput เดียวกัน พร้อม provenance ของการแปลง

R5 — Sensitivity labels ไม่ตรงทิศทางของพารามิเตอร์

profile_sets.py:379,427,431 เลือกปลาย low/high แบบเดียวกันทุก field แต่ accuracy/throughput สูงเป็นผลดี ขณะที่ tokens/latency/cost สูงเป็นผลเสีย

Probe configuration เดียวกัน: optimistic accuracy=81.35 แต่ pessimistic accuracy=82.35 จึงไม่ใช่ขอบเขต optimistic/pessimistic ของระบบอย่างถูกต้อง ต้องกำหนด direction ต่อ field และรักษาความสัมพันธ์ของ operating points

R6 — Forecast ใช้ข้อมูลใน epoch ที่กำลังตัดสินใจ

arrivals.py:78 ใช้ max/mean ของ samples ใน epoch นั้นโดยตรง จึงเป็น oracle demand inputs สำหรับ offline sizing ไม่ใช่การทำนายจากอดีตตาม §3.4 และ §4.7

ใช้ได้ในฐานะ oracle control แต่ต้องมี forecast path แยกและห้ามเรียกผลนี้ว่า online prediction performance

R7 — Experiment arms ยังไม่ตรง §4.3

compare.py:153 แยกการ solve ตาม workflow แต่รวมสอง SLOs ภายใน workflow เดียวกันแล้ว ส่วน paper Mkb Opt แยกทุก workflow-SLO pair

นอกจากนี้ headline test ใช้ ENERGY, epoch 0 และ derived_tiers ขณะที่ Table 2 เป็นผล trace 24 ชั่วโมง, COST objective และ good-tier 70/30 พร้อมระบบ runtime ของ paper

ควรคงสาม arms เป็น ablation ได้ แต่ชื่อและ setup ต้องตรง: four separate pairs; all-pairs joint $\mu=1$; all-pairs joint calibrated μ แล้วรายงานทั้ง snapshots และ resource-time integrals

R8 — Unbounded guard ให้เหตุผลผิด

objectives.py:116 และ joint.py:301 ปฏิเสธ accuracy objective ที่ไม่มี budget โดยอ้างว่าไม่ minimize n ทั้งที่ objective ที่ implement มี negative positive-cost penalty ต่อ n จึงไม่ใช่ unbounded objective โดยอัตโนมัติ อีกด้าน cost constraint Eq.6 ไม่มี n จึงไม่ได้สร้าง upper bound ต่อ fleet ตาม bounds_n_above()

Probe สร้าง objective เดียวกันโดยไม่ใช้ pre-solver guard: solver ให้ OPTIMAL, objective=76.2187267989 และ 72 GPUs ในกรณี Video QA accuracy-good การกำหนดว่าทดลอง accuracy-under-budget ต้องส่ง budget เป็น API policy ที่ทำได้ แต่ไม่ควรเรียกเหตุผลนั้นว่า mathematical unboundedness

R9 — Component boundaries ยังไม่ต่อถึง executable plan

ExecutionPreferences ถูกเก็บและส่งออกจาก orchestrator แต่ไม่มี consumer ใน optimizer; configuration enumeration มีเพียงสอง workflow IDs; prune_stt ถูกใช้ใน tests แต่ยังไม่มี deployment path ใช้ materialize executable workflow จาก configuration ที่ solver เลือก

Phase 1 และ Phase 2 จึงยังเป็นโมดูลที่ตรวจแยกกัน มากกว่า pipeline ที่ end-to-end แล้ว

R10 — ตัวเลือก diagnostic ปะปนใน executor library

Library มี PAPER/PAPER-FORM/INVENTED entries รวมถึง fixed interval segmenter และ LLM execution simulator พร้อม mock keyword selector การมี probes มีประโยชน์ แต่ selection บน catalog นี้ไม่ใช่หลักฐานพฤติกรรม orchestrator ของ paper

Baseline ควรมี catalog/fixture ที่ตรง workflow ที่ profile ส่วน adversarial alternatives อยู่ในชุดทดสอบแยก

ข้อสรุปในรายงานเดิมที่ต้องแก้หรือจำกัดความ

ข้อเดิม	ข้อสรุปที่หลักฐานรองรับ
Figure 12 ใช้ Llava และ TTFT ขาดทำให้การทดลอง scheduling ใช้ไม่ได้	ผิด ทั้งสองฉบับใช้ Gemma-3-27B; missing TTFT ของ Llava เป็น gap คนละเรื่อง
ไม่มี DAG terms ใน A.5 จึง Murakkab ทำ scheduling ไม่ได้	A.5 ไม่ระบุ scheduling ครบ; runtime scheduling ถูกอธิบายและประเมินใน Figure 12
STT knob ที่ลบ node เป็นสิ่งที่แสดงไม่ได้	กล่าวเกินหลักฐาน workflow-level configuration หรือ optional node/branch สามารถแทนได้; paper ไม่บอก lowering algorithm
Frame count เป็นทั้ง agent knob และ workflow configuration จึงขัดแย้งร้ายแรง	workflow configuration สามารถรวม agent knobs ได้; มีความคลุมเครือด้านศัพท์ แต่ไม่ใช่ logical impossibility
Model-profile constants บังคับให้เหลือหนึ่ง operating point ต่อ model/GPU/TP	ไม่ตามจากสมการ M เป็นเซต profiles และอาจ enumerate load operating points
ตารางแสดงหนึ่ง configuration ขัดกับ continuous routing	A.2/A.3 นอกว่าเป็น most commonly chosen configuration จึงไม่ขัดกับการสลับ configuration หรือ split traffic
n_m ไม่มี executor index แปลว่า paper บังคับใช้โมเดลเดียวทุก node	สูตร printed aggregate ไม่อธิบาย per-executor allocation ครบ; ยังสรุป actual author runtime แบบนั้นไม่ได้
100% Phi-4 on Video QA เป็นสิ่งที่ Murakkab จริงทำ	พิสูจน์ได้เฉพาะ unrestricted c×m ของ reproduction; paper §3.3.1 กล่าวถึง feasible profiles
mu=1 ให้ 0% เสมอ และ linear model ให้ 21.6% ไม่ได้	ไม่จริงทั่วไป fixed-model rounding ให้ 2 instances→1 instance หรือ 50% ได้; 0% เป็นผลเฉพาะรัน
mu=0.784 เป็นค่าที่ผู้เขียนใช้	เป็นค่าที่ reproduction fit จาก aggregate reduction ผู้เขียนไม่ได้รายงานค่านี้
mu uniform ต้องให้ GPU/energy/cost reduction เท่ากันเสมอ	ต้องมีสมมติฐานเพิ่มเรื่อง fixed allocation mix, integrality และการนับตลอดเวลา จึงไม่ใช่ข้อพิสูจน์ทั่วไป
Video tokens ต่ำสุด 250 และ latency ต่ำสุด 1.1s	profile ปัจจุบัน Llava F1 STT-on มี p90=215; lower bound ตาม table TPOT 0.0044 และ TTFT≥0 เป็น 0.946s
ทุกค่า MILP มาจาก table/figure เท่านั้น	มี external prices, derived assumptions และ oracle arrival aggregation; provenance นอกที่มาแต่ไม่พิสูจน์สมมติฐาน
Energy objective แยก models บน GPU type เดียวกันไม่ได้	เกิดจาก reproduction ใช้ energy ต่อ GPU type; A.5 ให้ e_m ต่อ model profile
A.5 ไม่มี instance-based provisioning	ผิด มี integer n_m อยู่แล้ว; สิ่งที่ขาดคือ resource/service detail และความเชื่อมโยงบางส่วน

ข้อที่ยังควรตรวจสอบต่อจริง: average allocation ไม่ถูก SLO/capacity-link แบบชัดเจน; configuration-model compatibility ไม่ถูกเขียน explicit; CPU/service-time encoding ไม่ครบ; mu estimation ไม่รายงาน; cost/energy units และ Cost_total ไม่ชัด; latency numbers ยังขัดกับการแทนค่าแบบที่ reproduction ใช้

เพิ่มเติมจาก algebra: demand upper bound $\alpha=1.15$ ไม่บังคับ 15% capacity headroom เมื่อ lower bound ยังเป็น λ ; accuracy objective หารด้วย arrival rate แต่ numerator อนุญาต allocation ได้ 1.15 เท่า Probe พบ average mass ratio ≈ 1.15 และ peak ratio ≈ 1.0 ต้องแยก headroom ออกจากจำนวน requests ที่นำไปคิดคะแนน

แนวทาง baseline และ improvement ที่ยังอยู่กับ Murakkab

เก็บผลสามประเภทแยกกัน:

1. Printed-A.5 audit: อ่านสมการตามตัวอักษร ใช้ตรวจ counterexamples และ omissions
2. Paper-behavior baseline: ทำ phases/components ที่ paper อธิบาย พร้อม assumption ledger สำหรับรายละเอียดที่ไม่ระบุ
3. Component improvement: เปลี่ยนหนึ่ง component จาก baseline ข้อ 2 โดยใช้ input, hardware/profile, SLO และ measurement เดียวกัน

Component	ทำ baseline ให้ครบก่อน	Improvement ที่ทดลองต่อได้
Specification/type checker	logical/executable distinction, optional STT และ dependencies ถูกต้อง	ตรวจ compatibility เร็วขึ้น ลด regeneration ด้วย feedback ที่เฉพาะจุด
Orchestrator/library	paper-grounded catalog และ execution adapters; fixture สำหรับ deterministic system experiment	candidate retrieval/caching และ semantic validation รัว success/retries/onboarding time
Workflow profiles	ผูก accuracy/tokens กับ workflow, prompt, executor/model ที่วัดจริง	selective profiling, uncertainty-aware pruning, drift-triggered refresh
Model profiles	เก็บ operating points ครบและ join TTFT/TPOT/TPS ที่ load เดียวกัน	เลือกจุดอย่างมี uncertainty margin และประหยัดจำนวน profiling runs
Configuration enumeration	ใช้ DAG/knob domains จริงและ materialize กลับเป็น executable plan ได้	dominance pruning หรือ hierarchical search โดยเทียบ feasible space เท่าเดิม
MILP	units/counts/domain/SLO correctness และ coherent mapping	warm start, pruning, resource/latency refinement ที่แยก ablation ชัดเจน
Forecast	historical EWMA, 60-min epoch และข้อมูลอนาคตใช้ประเมินเท่านั้น	adaptive error margin หรือเปลี่ยน predictor อย่างเดียว
Multiplexing	pair-separated vs jointly shared pools, ไม่ fit evaluation target	calibration จาก training traces ตาม pool/traffic แล้วประเมิน held-out
Registry/deployment	versioned executable plans, state และ readiness	ลด churn/transition overhead ด้วย plan reuse
Runtime scheduler	dependency-aware dispatch และ reproduce Figure 12 choices	HEFT/deadline policy ภายใต pool/profiles เดียวกัน; ไม่อ้างว่า original ใช้ HEFT
Auto-scaler	thresholds จาก profile curve, spare capacity, startup delay, early reopt	hysteresis/cooldown หรือ prediction-assisted scaling

การเพิ่ม CPU offloading, DAG awareness และ online scaling ครั้งแรกเป็น baseline completion เพราะ paper มีอยู่แล้ว การเปลี่ยนวิธีตัดสินใจภายใน component แล้วพิสูจน์ผลเหนือ baseline จึงเป็น improvement

ไม่จำเป็นต้องเริ่มด้วย multi-resource MILP ขนาดใหญ่หรือ M/G/c ทั้งระบบ การเปลี่ยนทุกชั้นพร้อมกันจะทำให้ attribution ยากและออกห่างจากเป้าหมายเปรียบเทียบทีละ component

ลำดับงานต่อที่เสนอ

1. แก้ factual claims และจัด gap ledger เป็น paper omission / reconstruction assumption / repo bug / incomplete feature / improvement hypothesis
2. แก้ GPU counting, cost units, TTFT joins, operating-point retention, sensitivity directions และ guard semantics พร้อม regression checks จาก quantities อีสระ
3. ต่อ logical workflow → compatible configuration/profile → executable workflow → registry → runtime ให้ครบด้วย deterministic simulator ที่ประกาศข้อจำกัดของข้อมูล
4. ตรวจ known-answer cases รวม Figure 12; ย่อารจนจบทุก milestone จึงตรวจ numerical consistency

5. ดึงการทดลอง §4.3 ให้ตรงทั้ง policy isolation, objective, SLO, trace horizon และ startup/auto-scaling assumptions

6. เลือกหนึ่ง improvement ทดลอง ablation; ใช้ held-out traces/datasets และรายงาน accuracy/latency/SLO violations/resource-time/energy/cost/churn

Simulator ใช้เรียนรู้และตรวจ control logic ได้ แต่ผล SLO จริง, batching, energy และ end-to-end quality ต้องอาศัย measurements หรือถูกระบุว่าเป็น model-based results อย่างชัดเจน

ณ จบ review นี้ยังไม่ได้แก่ production implementation, design policy หรือ milestone status มีเพียงเอกสาร review นี้และ diagnostic artifacts ใน tmp/